AutoNAT을 통한 NAT 탐지 메커니즘 분석

개요

AutoNAT(Automatic Network Address Translation Detection)은 libp2p에서 피어가 자신이 NAT 뒤에 있는지, 아니면 공개적으로 접근 가능한지를 자동으로 탐지하는 프로토콜입니다. 이 문서는 Rust libp2p의 AutoNAT 구현체를 상세히 분석하여 NAT 탐지 메커니즘의 동작 방식을 설명합니다.

AutoNAT 프로토콜 개요

기본 개념

AutoNAT은 다음과 같은 기본 원리로 동작합니다:

1. **클라이언트**: NAT 상태를 확인하고 싶은 피어
2. **서버**: 클라이언트의 요청에 따라 다이얼백(dial-back)을 수행하는 피어
3. **다이얼백 요청**: 클라이언트가 서버에게 자신의 주소로 연결을 시도해달라고 요청
4. **NAT 상태 판단**: 다이얼백 성공/실패 여부에 따라 NAT 상태 결정

프로토콜 버전

libp2p AutoNAT은 두 가지 버전을 지원합니다:

- **v1**: `/libp2p/autonat/1.0.0` - 현재 널리 사용되는 버전
- **v2**: `/libp2p/autonat/2/dial-request`, `/libp2p/autonat/2/dial-back` - 개선된 새 버전

v1 구현 분석

핵심 구조체

NatStatus

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub  enum NatStatus {
Public(Multiaddr), // 공개 주소와 함께 공개 상태
Private, // NAT 뒤에 있는 상태
Unknown, // 알 수 없는 상태
}
```

Config

```
pub struct Config {
pub timeout: Duration, // 요청 타임아웃
pub boot_delay: Duration, // 초기 프로브 지연
pub refresh_interval: Duration, // 상태 갱신 간격
pub retry_interval: Duration, // 재시도 간격
pub throttle_server_period: Duration, // 서버 스로틀링 기간
pub use_connected: bool, // 연결된 피어를 서버로 사용
pub confidence_max: usize, // 최대 신뢰도
pub max_peer_addresses: usize, // 최대 피어 주소 수
pub throttle_clients_global_max: usize, // 전역 클라이언트 스로틀링
pub throttle_clients_peer_max: usize, // 피어별 클라이언트 스로틀링
pub throttle_clients_period: Duration, // 클라이언트 스로틀링 기간
```

```
pub only_global_ips: bool, // 글로벌 IP만 사용

}
```

클라이언트 동작 (AsClient)

1. 자동 프로브 스케줄링

```
pub(crate) fn poll_auto_probe(&mut self, cx: &mut Context<'_>) -> Poll<OutboundP

match self.schedule_probe.poll_unpin(cx) {

Poll::Ready(()) => {

self.schedule_probe.reset(self.config.retry_interval);

let addresses = self

.other_candidates

.iter()

.chain(self.listen_addresses.iter())

.cloned()

.collect();


let probe_id = self.probe_id.next();

let event = match self.do_probe(probe_id, addresses) {

Ok(peer) => OutboundProbeEvent::Request { probe_id, peer },

Err(error) => {

self.handle_reported_status(NatStatus::Unknown);

OutboundProbeEvent::Error {
```

```
probe_id,
peer: None,
error,
}
}
};
Poll::Ready(event)
}
Poll::Pending => Poll::Pending,
}
}
```

2. 서버 선택 알고리즘

```
fn random_server(&mut self) -> Option<PeerId> {
// 스로틀링된 서버 목록 업데이트
let i = self.throttled_servers.partition_point(|(_, time)| {
*time + self.config.throttle_server_period < Instant::now()
});
self.throttled_servers.drain(..i);

let mut servers: Vec<&PeerId> = self.servers.iter().collect();
```

```
if self.config.use_connected {
servers.extend(self.connected.iter().filter_map(|(id, addrs)| {
// 릴레이되지 않고 글로벌 주소를 가진 연결만 필터링
addrs.values().any(|a| a.is_some()).then_some(id)
}));
}

// 스로틀링된 서버 제외
servers.retain(|s| !self.throttled_servers.iter().any(|(id, _)| s == &id));

servers.choose(&mut thread_rng()).map(|&&p| p)
}
```

3. 프로브 실행

```
fn do_probe(
&mut self,
probe_id: ProbeId,
addresses: Vec<Multiaddr>,
) -> Result<PeerId, OutboundProbeError> {
let _ = self.last_probe.insert(Instant::now());
if addresses.is_empty() {
return Err(OutboundProbeError::NoAddresses);
```

```
}

let server = self.random_server().ok_or(OutboundProbeError::NoServer)?;

let request_id = self.inner.send_request(
&server,
DialRequest {
peer_id: self.local_peer_id,
addresses,
},
);
self.throttled_servers.push((server, Instant::now()));
self.ongoing_outbound.insert(request_id, probe_id);
Ok(server)
}
```

4. 상태 처리 로직

```
fn handle_reported_status(&mut self, reported_status: NatStatus) -> Option<NatSt
self.schedule_next_probe(self.config.retry_interval);

if matches!(reported_status, NatStatus::Unknown) {
return None;
}
```

```
if reported_status == *self.nat_status {
// 같은 상태 보고 시 신뢰도 증가
if *self.confidence < self.config.confidence_max {
*self.confidence += 1;
}
// 최대 신뢰도 도달 시 갱신 간격으로 변경
if *self.confidence >= self.config.confidence_max {
self.schedule_next_probe(self.config.refresh_interval);
}
return None;
}


if reported_status.is_public() && self.nat_status.is_public() {
// 다른 공개 주소 보고 시 주소만 변경
*self.nat_status = reported_status;
return None;
}
if *self.confidence > 0 {
// 다른 상태 보고 시 신뢰도 감소
*self.confidence -= 1;
return None;
}
```

```
// 신뢰도가 0이면 상태 변경
let old_status = self.nat_status.clone();
*self.nat_status = reported_status;
Some(old_status)
}
```

서버 동작 (AsServer)

1. 인바운드 요청 처리

```
fn resolve_inbound_request(
&mut self,
sender: PeerId,
request: DialRequest,
) -> Result<Vec<Multiaddr>, (String, ResponseError)> {
// 스로틀링 클라이언트 목록 업데이트
let i = self.throttled_clients.partition_point(|(_, time)| {
*time + self.config.throttle_clients_period < Instant::now()
});
self.throttled_clients.drain(..i);

// 피어 ID 검증
if request.peer_id != sender {
```

```
return Err(("peer id mismatch".to_string(), ResponseError::BadRequest));
}

// 중복 요청 검사
if self.ongoing_inbound.contains_key(&sender) {
return Err(("dial-back already ongoing".to_string(), ResponseError::DialRefused)
}

// 글로벌 스로틀링 검사
if self.throttled_clients.len() >= self.config.throttle_clients_global_max {
return Err(("too many total dials".to_string(), ResponseError::DialRefused));
}

// 피어별 스로틀링 검사
let throttled_for_client = self
.throttled_clients
.iter()
.filter(|(p, _)| p == &sender)
.count();

if throttled_for_client >= self.config.throttle_clients_peer_max {
return Err(("too many dials for peer".to_string(), ResponseError::DialRefused));
}
```

```
// 관찰된 주소 획득
let observed_addr = self
.connected
.get(&sender)
.expect("Peer is connected.")
.values()
.find_map(|a| a.as_ref())
.ok_or_else(|| {
("refusing to dial peer with blocked observed address".to_string(),
ResponseError::DialRefused)
})?;

let mut addrs = Self::filter_valid_addrs(sender, request.addresses, observed_add
addrs.truncate(self.config.max_peer_addresses);

if addrs.is_empty() {
return Err(("no dialable addresses".to_string(), ResponseError::DialRefused));
}

Ok(addrs)
}
```

2. 주소 필터링 및 변환

```
fn filter_valid_addrs(
peer: PeerId,
demanded: Vec<Multiaddr>,
observed_remote_at: &Multiaddr,
) -> Vec<Multiaddr> {
let Some(observed_ip) = observed_remote_at
.into_iter()
.find(|p| matches!(p, Protocol::Ip4(_) | Protocol::Ip6(_)))
else {
return Vec::new();
};


let mut distinct = HashSet::new();
demanded
.into_iter()
.filter_map(|addr| {
// 요청된 IP를 관찰된 IP로 교체
let i = addr
.iter()
.position(|p| matches!(p, Protocol::Ip4(_) | Protocol::Ip6(_)))?;
let mut addr = addr.replace(i, |_| Some(observed_ip.clone()))?;
```

```
let is_valid = addr.iter().all(|proto| match proto {
Protocol::P2pCircuit => false, // 릴레이 주소 거부
Protocol::P2p(peer_id) => peer_id == peer,
_ => true,
});

if !is_valid {
return None;
}
if !addr.iter().any(|p| matches!(p, Protocol::P2p(_))) {
addr.push(Protocol::P2p(peer))
}
// 중복 주소 제거
distinct.insert(addr.clone()).then_some(addr)
})
.collect()
}
```

3. 다이얼백 결과 처리

```
pub(crate) fn on_outbound_connection(
&mut self,
peer: &PeerId,
```

```
address: &Multiaddr,
) -> Option<InboundProbeEvent> {
let (_, _, addrs, _) = self.ongoing_inbound.get(peer)?;


// 다이얼된 주소가 요청된 주소 중 하나인지 확인
if !addrs.contains(address) {
return None;
}


let (probe_id, _, _, channel) = self.ongoing_inbound.remove(peer).unwrap();
let response = DialResponse {
result: Ok(address.clone()),
status_text: None,
};
let _ = self.inner.send_response(channel, response);


Some(InboundProbeEvent::Response {
probe_id,
peer: *peer,
address: address.clone(),
})
}
```

프로토콜 메시지 구조

DialRequest

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct DialRequest {
pub peer_id: PeerId, // 클라이언트 피어 ID
pub addresses: Vec<Multiaddr>, // 다이얼백할 주소 목록
}
```

DialResponse

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct DialResponse {
pub status_text: Option<String>, // 상태 메시지
pub result: Result<Multiaddr, ResponseError>, // 성공 시 주소, 실패 시 에러
}
```

ResponseError

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub enum ResponseError {
DialError, // 다이얼 실패
DialRefused, // 다이얼 거부
```

```
BadRequest, // 잘못된 요청

InternalError, // 내부 오류

}
```

NAT 탐지 알고리즘

1. 상태 전환 로직

```
Unknown → Public: 다이얼백 성공 시

Unknown → Private: 다이얼백 실패 시

Public → Private: 연속된 다이얼백 실패 시 (신뢰도 기반)

Private → Public: 연속된 다이얼백 성공 시 (신뢰도 기반)
```

2. 신뢰도 시스템

- **신뢰도 증가**: 현재 상태와 일치하는 결과 시
- **신뢰도 감소**: 현재 상태와 다른 결과 시
- **상태 변경**: 신뢰도가 0에 도달했을 때

3. 프로브 스케줄링

```
// 초기 지연

boot_delay: Duration::from_secs(15)
```

```
// 재시도 간격 (낮은 신뢰도 시)
retry_interval: Duration::from_secs(90)


// 갱신 간격 (높은 신뢰도 시)
refresh_interval: Duration::from_secs(15 * 60)
```

보안 및 DoS 방어

1. 스로틀링 메커니즘

클라이언트 측 서버 스로틀링

```
throttle_server_period: Duration::from_secs(90)
```

서버 측 클라이언트 스로틀링

```
throttle_clients_global_max: 30, // 전역 최대 동시 요청
throttle_clients_peer_max: 3, // 피어별 최대 동시 요청
throttle_clients_period: Duration::from_secs(1)
```

2. 주소 검증

- **IP 교체**: 클라이언트가 요청한 IP를 서버가 관찰한 IP로 교체
- **릴레이 주소 거부**: P2pCircuit 프로토콜을 포함한 주소 거부
- **글로벌 IP 필터링**: `only_global_ips` 설정 시 사설 IP 거부

3. 요청 검증

- **피어 ID 일치**: 요청의 피어 ID와 연결의 피어 ID 일치 확인
- **중복 요청 방지**: 동일 피어의 동시 요청 거부
- **주소 개수 제한**: `max_peer_addresses` 로 주소 개수 제한

글로벌 IP 판단 로직

IPv4 글로벌 IP 판단

```
fn is_global_ip(&self) -> bool {
// RFC 1918 사설 주소 제외
!self.is_private()
&& !self.is_loopback()
&& !self.is_link_local()
&& !self.is_broadcast()
&& !self.is_documentation()
&& !is_shared(self) // 100.64.0.0/10
&& !is_reserved(self) // 240.0.0.0/4
&& !is_benchmarking(self) // 198.18.0.0/15
&& self.octets()[0] != 0 // 0.0.0.0/8 제외
}
```

IPv6 글로벌 IP 판단

```
fn is_global_ip(&self) -> bool {
match is_multicast_scope_global(self) {
Some(true) => true, // 글로벌 멀티캐스트
None => is_unicast_global(self), // 글로벌 유니캐스트
_ => false,
}
}
```

이벤트 처리 흐름

클라이언트 이벤트

```
pub enum OutboundProbeEvent {
Request { probe_id: ProbeId, peer: PeerId },
Response { probe_id: ProbeId, peer: PeerId, address: Multiaddr },
Error { probe_id: ProbeId, peer: Option<PeerId>, error: OutboundProbeError },
}
```

서버 이벤트

```
pub enum InboundProbeEvent {
Request { probe_id: ProbeId, peer: PeerId, addresses: Vec<Multiaddr> },
Response { probe_id: ProbeId, peer: PeerId, address: Multiaddr },
```

```
Error { probe_id: ProbeId, peer: PeerId, error: InboundProbeError },
}
```

상태 변경 이벤트

```
Event::StatusChanged {
old: NatStatus,
new: NatStatus,
}
```

실제 사용 예제

클라이언트 설정

```
let auto_nat = autonat::Behaviour::new(
local_peer_id,
autonat::Config {
retry_interval: Duration::from_secs(10),
refresh_interval: Duration::from_secs(30),
boot_delay: Duration::from_secs(5),
throttle_server_period: Duration::ZERO,
only_global_ips: false,
..Default::default()
```

```
},
);

// 서버 추가
auto_nat.add_server(server_peer_id, Some(server_address));
```

서버 설정

```
let auto_nat = autonat::Behaviour::new(
local_peer_id,
autonat::Config {
only_global_ips: false,
..Default::default()
},
);
```

v2 개선사항

v2 프로토콜은 다음과 같은 개선사항을 제공합니다:

1. **새 포트 할당**: 서버가 항상 새로운 포트로 다이얼백하여 홀 펀칭으로 인한 거짓 양성 방지
2. **DoS 방어**: 클라이언트가 서버보다 더 많은 데이터를 전송하도록 하여 DoS 공격 방어
3. **분리된 프로토콜**: 다이얼 요청과 다이얼백을 별도 프로토콜로 분리

결론

AutoNAT은 libp2p 네트워크에서 피어의 NAT 상태를 자동으로 탐지하는 중요한 프로토콜입니다. 신뢰도 기반 상태 관리, 스로틀링을 통한 DoS 방어, 그리고 정교한 주소 검증 메커니즘을 통해 안정적이고 정확한 NAT 탐지를 제공합니다. v2에서는 보안과 정확성이 더욱 개선되어 프로덕션 환경에서의 사용에 적합합니다.